

Objects, Classes & Inheritance

Pre-Assessment 4 · 8 questions · 29 points

Name: _____

Date: _____

Score: / 29

This is a **pre-assessment**. It shows what you already know **before** we start this unit, so it is fine if some questions cover things you have not learned yet. Try every question you can. If you have no idea at all, write “**not yet**” and move on.

Work on your own, without a computer. You have about **40 minutes**. When a question asks what a program prints, write exactly what Python would print, one line of output per answer line. If the code would crash, write the name of the error (for example, `TypeError`). Show your thinking; partial credit is given.

1. Objects everywhere

3 points

(a) What does `print(type([1, 2]))` show?

(1 pt)

(b) Which built-in function lists all the methods an object has?

(1 pt)

(c) Finish the comparison: a **class** is like a cookie cutter, and an **object** is like a _____.

(1 pt)

2. Pet class

4 points

```
class Pet:
    def __init__(self, name, species):
        self.name = name
        self.species = species
        self.energy = 5

    def play(self):
        self.energy = self.energy - 2
        return f"{self.name} plays!"

    def nap(self):
        self.energy = self.energy + 3

a = Pet("Mochi", "cat")
b = Pet("Rex", "dog")
print(a.play())
a.play()
b.nap()
print(a.energy, b.energy)
```

(a) Write exactly what the program prints.

(2 pts)

(b) When `a.play()` runs, what does `self` refer to?

(1 pt)

(c) How many `Pet` objects (instances) does the program create?

(1 pt)

3. Readable objects

2 points

```
class Book:
    def __init__(self, title, pages):
        self.title = title
        self.pages = pages

    def __str__(self):
        return f"{self.title} ({self.pages} pages)"

print(Book("Holes", 233))
```

(a) What does this print?

(1 pt)

(b) What would `print` show if the class had **no** `__str__` method?

(1 pt)

4. Team Builder

4 points

```
class Player:
    def __init__(self, name):
        self.name = name

class Team:
    max_size = 3

    def __init__(self, name):
        self.name = name
        self.players = []

    def add(self, player):
        if len(self.players) < Team.max_size:
            self.players.append(player)
            return True
        return False

t = Team("Comets")
for n in ["Ana", "Ben", "Cy", "Dee"]:
    print(n, t.add(Player(n)))
print(len(t.players), t.players[0].name)
```

(a) Write exactly what the program prints.

(3 pts)

(b) Is `max_size` a class attribute or an instance attribute? What does that mean?

(1 pt)

5. Animal family

4 points

```
class Animal:
    def __init__(self, name):
        self.name = name
        self.sound = "..."

    def speak(self):
        return f"{self.name} says {self.sound}"

class Dog(Animal):
    def __init__(self, name, trick):
        super().__init__(name)
        self.sound = "Woof"
        self.trick = trick

class Cat(Animal):
    def speak(self):
        return f"{self.name} ignores you"

pets = [Dog("Rex", "roll over"), Cat("Luna"), Animal("Blob")]
for p in pets:
    print(p.speak())
print(pets[0].trick)
```

Write exactly what the program prints.

6. isinstance()

3 points

Use the `pets` list from Question 5. Write **True** or **False**.

Expression	Result
<code>isinstance(pets[0], Dog)</code>	
<code>isinstance(pets[0], Animal)</code>	
<code>isinstance(pets[1], Dog)</code>	

7. Bug Bounty

3 points

This class has **two** bugs.

```
class Monster:
    def __init__(name, hp):
        self.name = name
        self.hp = hp

    def take_hit(self, damage):
        hp = self.hp - damage

m = Monster("Blaze", 20)
m.take_hit(5)
print(m.hp)
```

(a) Find and fix both bugs. Write the corrected lines.

(2 pts)

(b) After your fixes, what does the program print?

(1 pt)

8. Monster Battle Arena

6 points

```
class Monster:
    def __init__(self, name, hp):
        self.name = name
        self.hp = hp

    def attack(self):
        return 5
```

Write a class `FireMonster` that **inherits** from `Monster`:

- `__init__(self, name, hp)` uses `super()` to set the name and hp, and also sets `fire_power` to 3
- it **overrides** `attack()` so it returns `5 + fire_power`
- it has a `__str__` method so `print(FireMonster("Blaze", 20))` shows `Blaze (HP 20)`

END OF ASSESSMENT